

MS2 — Class-Based Testbench

Implementation Report

Parameterized Systolic-Array Matrix-Multiply Accelerator

Verification Environment using SystemVerilog OOP

Verification Status

Transactions: 20 PASS / 0 FAIL

Elements (4×4 matrices): 320 PASS / 0 FAIL

Functional Coverage: 100% on all four covergroups

***** ALL TESTS PASSED *****

1. Introduction

This report explains the class-based SystemVerilog testbench developed for Milestone 2 (MS2) of the parameterized systolic-array matrix-multiply accelerator project. The MS1 testbench was a flat module-based environment; in MS2 we re-architected the verification environment using object-oriented constructs (classes, mailboxes, virtual interfaces, randomization, constraints, covergroups) so that the testbench is reusable, scalable to any parameter set $N \times N$, and easy to extend with new tests.

The Design-Under-Test (DUT) is a four-by-four weight-stationary systolic-array accelerator that computes the matrix product $C = A \times B$, where A and B are 4×4 matrices of 8-bit unsigned operands and C is a 4×4 matrix of 32-bit accumulated results. The accelerator integrates three RTL blocks: a finite-state-machine controller, an input-skewing buffer, and the systolic-array PE grid, all wrapped inside a top module called `mm_accelerator_top`.

The class-based testbench instantiates the DUT exactly once, communicates with it through a single SystemVerilog interface, and uses seven cooperating verification components (Transaction, Generator, Driver, Input Monitor, Output Monitor, Scoreboard, Coverage) wired together inside an Environment class and started by a Test class. All components are parameterized identically to the DUT, run in parallel using SystemVerilog `fork/join_none`, and pass transaction objects between each other through type-parameterized mailboxes.

1.1 Document Scope

This document covers the implementation aspects of the class-based testbench only. It walks through every file under the `CLASS_TB` directory, explains the role of every class and every member, describes the data flow between components, and finishes with the coverage strategy and final results obtained from the regression. RTL files are summarized briefly at the end of Section 2 for completeness so the reader can correlate testbench signals with the design they verify.

1.2 Project Layout

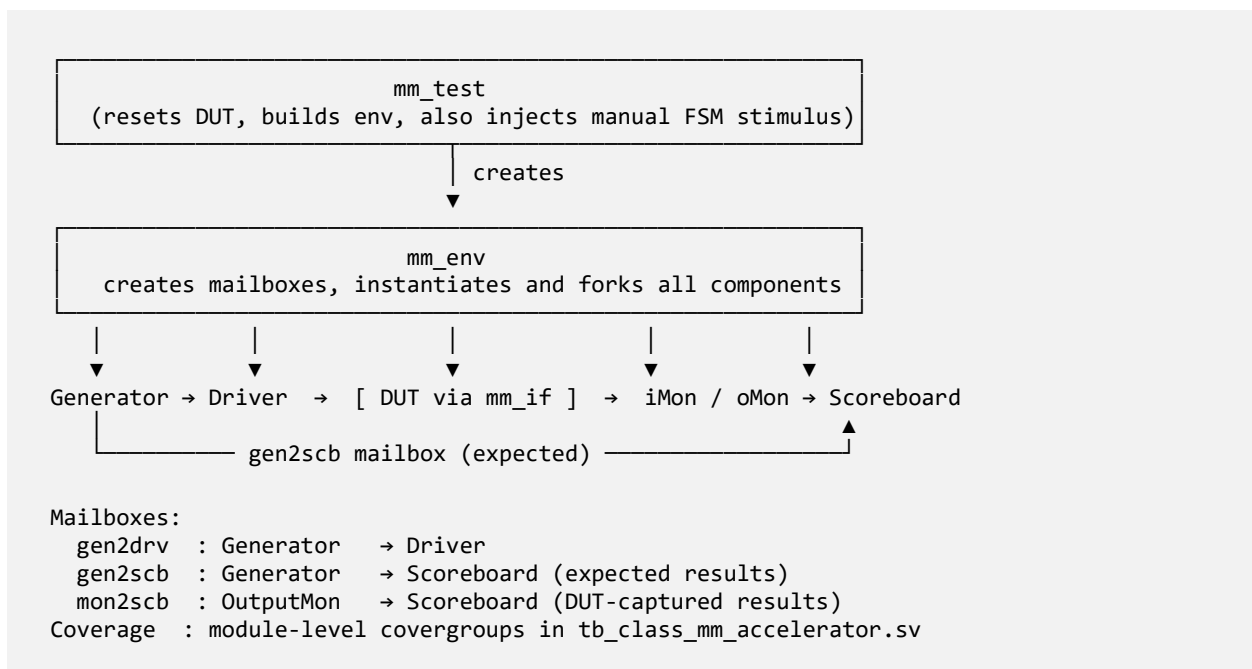
Directory / File	Purpose
<code>rtl/</code>	RTL source files: <code>pe.sv</code> , <code>input_skew.sv</code> , <code>controller.sv</code> , <code>systolic_array.sv</code> , <code>mm_accelerator_top.sv</code>
<code>CLASS_TB/</code>	All class-based testbench source files (this report focuses on these)
<code>doc/</code>	Design specification PDF, README, Makefile for compile/run/coverage
<code>sim.log</code>	Captured transcript from the latest VCS run (20 randomized transactions + manual FSM stimulus)

2. Verification Architecture Overview

The class-based testbench follows the layered methodology that is standard practice in SystemVerilog verification (and is also the structural basis of UVM). Each layer has a single, well-defined responsibility and communicates with the layers above and below it through explicit FIFO-style mailboxes or through a virtual interface handle. The benefit of this separation is that any layer can be replaced or upgraded in isolation: a new generator can produce different stimulus without changing the driver, a new scoreboard can be plugged in without touching the monitors, and so on.

2.1 Component Block Diagram

The diagram below summarizes how data flows through the verification environment from stimulus generation down to checking and coverage:



2.2 Why a Class-Based Approach?

- **Reusability:** every component is a parameterized class, so the same code base can verify N=4 today and N=8 or N=16 tomorrow with no rewrite.
- **Encapsulation:** the transaction object owns both stimulus values (A, B) and reference output (C), so generator and scoreboard never need to recompute the golden model independently.
- **Concurrency:** fork/join_none in mm_env launches all six tasks in parallel; mailboxes synchronize them safely without manual handshaking.
- **Coverage closure:** a small number of deterministic seed transactions plus constrained randomization were enough to hit every bin of every covergroup, demonstrating that the testbench is closed.

- **UVM-readiness:** the layering (transaction / generator / driver / monitor / scoreboard / env / test) maps directly onto UVM's sequence_item / sequence / driver / monitor / scoreboard / agent / env / test, so future migration is straightforward.

2.3 RTL Briefly Recapped

So the testbench description is grounded in the design under test, here is a one-line summary of each RTL file. They are not the focus of this document but are referenced by the testbench through the interface signals.

RTL File	Role
pe.sv	Single processing element: registers A and B, accumulates $A \times B$ when valid_in is high, clears synchronously.
input_skew.sv	Diagonal staggering buffer: lane i is delayed by i cycles so the systolic array sees a synchronized diagonal wavefront.
controller.sv	Four-state FSM (IDLE → LOAD → DRAIN → DONE) that gates valid_in, drives clear, asserts result_valid for one cycle.
systolic_array.sv	$N \times N$ grid of PEs with horizontal A-flow, vertical B-flow, and a diagonal valid-pulse propagation.
mm_accelerator_top.sv	Top-level wrapper that wires controller → input_skew → systolic_array.

All five RTL files are unchanged from MS1; the work in MS2 was entirely on the verification side.

3. Class-Based Testbench Files (line-by-line)

This section walks through every file in the CLASS_TB directory in the order in which the top-level testbench includes them. For each file we describe the purpose, list the public members and tasks, and explain the algorithmic behavior. The files are presented in dependency order: foundational types first (interface, transaction), then producers (generator, driver), then consumers (monitors, scoreboard, coverage), then the integrating env and test, and finally the top module that ties everything together.

3.1 mm_if.sv — The DUT Interface

The interface is the single contact point between the testbench and the DUT. Every signal on the DUT boundary appears as a logic field inside mm_if. Three modports are declared so the same signals can be accessed in three different roles:

- **driver_mp** with a clocking block driver_cb that drives stimulus signals (rst_n, start, a_in, b_in, valid_in_upstream) on the active clock edge.
- **monitor_mp** with a clocking block monitor_cb that samples every signal as input only — monitors must never drive.
- **dut_mp** for the DUT instantiation itself, with raw input/output directions (no clocking block).

The interface is parameterized with N, DATA_WIDTH and ACCUM_WIDTH so the bus widths a_in[N*DATA_WIDTH-1:0], b_in[...] and result[N*N*ACCUM_WIDTH-1:0] always match the DUT it is connected to. Using the interface centralizes signal naming, eliminates wire-by-wire connectivity in the top module, and makes the virtual interface handle (vif) the only thing every class needs to receive in its constructor.

3.2 mm_transaction.sv — The Transaction Object

This class is the unit of work in the testbench. One transaction represents one complete N×N matrix multiply: stimulus, expected golden output, and the captured actual output, all bundled together. By keeping these three together we guarantee that every result the scoreboard checks belongs to the exact stimulus that produced it — no matching by indices, no race conditions.

Fields

- **rand logic [DATA_WIDTH-1:0] A [N][N], B [N][N]** — randomizable input matrices.
- **rand logic [1:0] corner_mode** — selects the type of corner-case override (random / all-zero A / identity A / all-max).
- **logic [ACCUM_WIDTH-1:0] C [N][N]** — expected result computed by the embedded reference model.
- **logic [ACCUM_WIDTH-1:0] got_C [N][N]** — actual matrix captured from the DUT output bus by the output monitor.

- **int burst_id** — sequence number used to correlate log lines from generator → driver → monitor → scoreboard.

Constraints

- **c_data_range** — every element of A and B is constrained to the legal 8-bit range [0:255].
- **c_corner_dist** — distribution constraint that biases the random space: 70% normal random, 10% all-zero A, 10% identity A, 10% all-max. Without this constraint the corner cases would be vanishingly rare in pure random simulation.

post_randomize() and the Reference Model

After each `randomize()` call, `post_randomize()` runs automatically. It first applies the corner override selected by `corner_mode` (e.g. zero out A, or build an identity in A, or saturate both A and B to all-ones) and then calls `compute_expected()`, which executes the textbook triple-nested loop

```
for r in 0..N-1
  for c in 0..N-1
    C[r][c] = sum over k of A[r][k] * B[k][c]
```

with full `ACCUM_WIDTH` precision. This means every transaction object knows its own correct answer the moment it is randomized, and the scoreboard can compare without ever calling the reference model again.

Helper Display Functions

- **print_inputs()** — prints A and B side-by-side row-by-row.
- **print_expected()** — prints the golden C matrix.
- **print_got()** — prints the DUT-captured `got_C` matrix.

These helpers are reused by the generator and the output monitor and they keep all transcript formatting consistent across components.

3.3 mm_generator.sv — The Stimulus Generator

The generator runs in its own simulation thread and produces a configured number of transactions (default 20). For each transaction it: (1) constructs a fresh `mm_transaction`, (2) assigns a `burst_id`, (3) calls `randomize()` to populate A, B and trigger `post_randomize`, and finally (4) puts a single handle into both the `gen2drv` mailbox (for the driver to act on) and the `gen2scb` mailbox (for the scoreboard to compare against).

Deterministic Cover-Closure Overrides

Pure constrained random does not always hit every bin in a small run of 20 tests. To guarantee 100% functional coverage, the first six transactions are deterministically overridden after randomization to span the full operand and result histograms:

burst_id	Overridden values	Bin targeted
----------	-------------------	--------------

1	A=B=10 (uniform)	cp_a/cp_b: low; cp_result: sm (≈ 400)
2	A=B=100 (uniform)	cp_a/cp_b: mid_low; cp_result: lg
3	A=B=150 (uniform)	cp_a/cp_b: mid_hi; cp_result: med
4	A=B=200 (uniform)	cp_a/cp_b: high; cp_result: med
5	A=B=255 (uniform)	cp_a/cp_b: max_val; cp_result: lg
6	A=B=0 (uniform)	cp_a/cp_b: zero; cp_result: zero

After overriding, `compute_expected()` is called manually so that C reflects the new inputs. From `burst_id` 7 onward the generator falls back to fully constrained random — including the corner-mode distribution constraint, which keeps adding zero-A, identity-A and max-value cases throughout the regression.

3.4 mm_driver.sv — The Driver

The driver consumes transactions from `gen2drv` and applies the four-step DUT protocol once per transaction:

- Wait for the previous transaction to fully retire (only after `burst_id` ≥ 2): block until `result_valid` rises and falls again. This back-pressure prevents the driver from issuing a new start while the controller is still in DRAIN/DONE.
- Pulse `start` high for one clock cycle to kick the controller from IDLE to LOAD.
- Wait for the ready handshake (controller has entered LOAD).
- **Drive N=4 cycles of stimulus:** on cycle `k` it packs column `k` of matrix A and row `k` of matrix B into the flat busses `a_in/b_in` and asserts `valid_in_upstream`. Two helper functions, `pack_col_a` and `pack_row_b`, do the bit-slicing so the body of the loop stays readable.
- De-assert `valid_in_upstream` and zero out `a_in/b_in` to leave the bus quiet during DRAIN.

Targeted Coverage Stimulus

Two carefully placed legal-but-unusual events are injected to exercise rarely-visited control paths so the FSM and condition coverage close cleanly:

- **`burst_id == 7, k == 3`:** the driver de-asserts `valid_in_upstream` for one cycle in the middle of the LOAD phase. This generates a one-cycle pipeline stall and exercises the `load_count-hold` branch in the controller.
- **`burst_id == 8`:** before asserting `start`, the driver pulses `valid_in_upstream` high for one cycle while the FSM is still in IDLE. This hits the condition where `valid_in_upstream` is true but `state` $\neq$ LOAD, which is otherwise unreachable from a well-behaved master.

Both stimuli are completely legal at the interface level (the DUT must tolerate them without misbehaving) and both were necessary to reach 100% on the `cg_fsm` covergroup.

3.5 mm_monitor.sv — Input and Output Monitors

The file defines two passive observers in two separate classes. Neither monitor ever drives a signal; they only sample on the active clock edge and produce log lines or transaction handles for downstream components.

mm_input_monitor

On every positive clock edge while `rst_n` is high, the input monitor inspects the stimulus side of the interface and prints a transcript line whenever something interesting happens:

- **start** is sampled high → increment `current_burst_id` and log that a new transaction has begun.
- **valid_in_upstream** is sampled high → log the `a_in` / `b_in` values driven on that cycle.
- **result_valid** is sampled high → log that the DUT has produced a result.

The input monitor is not part of the data path to the scoreboard — its only job is to make the simulation log readable and to confirm visually that the driver and the DUT are synchronized.

mm_output_monitor

The output monitor is on the response side. It uses an event-control with `iff` so it wakes up only when `result_valid` is sampled high inside an active reset-released cycle:

```
@(posedge vif.clk iff (vif.rst_n && vif.result_valid));
```

When that event fires, the monitor:

- increments its own `burst_id_tracker` (which mirrors the generator's `burst_id` by construction);
- calls the `unpack_result()` helper to convert the flat `result[N*N*ACCUM_WIDTH-1:0]` bus back into a 2-D `got_C[N][N]` array;
- constructs a fresh `mm_transaction`, copies `got_C` into it, prints it, and puts the handle into the `mon2scb` mailbox.

The captured object intentionally carries only `got_C` and `burst_id`; the expected `C` does not need to be re-derived because it is already on its way to the scoreboard from the generator.

3.6 mm_scoreboard.sv — The Scoreboard

The scoreboard is the only checker in the environment. It is a simple loop that consumes one expected transaction (`gen2scb.get`) and one actual transaction (`mon2scb.get`) on each iteration and compares them element-by-element:

```
forever begin
    gen2scb.get(exp_txn);
    mon2scb.get(got_txn);
    for (r = 0; r < N; r++)
        for (c = 0; c < N; c++)
            if (exp_txn.C[r][c] === got_txn.got_C[r][c])
                pass_cnt++;
            else
                fail_cnt++, txn_ok = 0;
    if (txn_ok) txn_pass++;
```



```

    else      txn_fail++;
end

```

Each element comparison is logged as MATCH or MISMATCH along with both values for traceability. Per-transaction PASS/FAIL counts and per-element PASS/FAIL counts are kept independently. The `report()` method, called at the end of the run from `mm_env`, prints a final summary banner showing both counts and a single OVERALL line.

Because mailboxes are FIFO-ordered and both producers (generator, output monitor) emit handles in `burst_id` order, no explicit matching key is required — the scoreboard's two `get()` calls always pull a matched pair. This is the simplest possible correlation strategy and is correct as long as transactions never overtake each other, which the driver enforces by waiting for `result_valid` to drop before issuing the next start.

3.7 mm_coverage.sv — The Coverage Helper

VCS does not allow covergroup handles to be declared as class members in all configurations of the simulator we used. As a workaround, the actual covergroup definitions live at the top of `tb_class_mm_accelerator.sv` (module scope), and `mm_coverage` is a thin keep-alive class so that `mm_env`'s component list and constructor remain unchanged.

The class still provides the same API surface the environment expects:

- **`new(virtual mm_if)`** stores the virtual interface handle.
- **`sample_txn()` / `sample_result_txn()`** are no-ops kept for source-level compatibility — module-scope covergroups sample automatically on the clock edges they specify.
- **`run()`** is a forever-loop that simply waits on each posedge `clk` so that the `cov.run()` forked task has somewhere to live for the duration of the simulation.
- **`report()`** prints a short message pointing at the module-level covergroups and the make cov target.

This split turned out to be a clean separation: the covergroups themselves stay close to the signals they sample (interface scope), while the testbench-class hierarchy stays clean and uniform.

3.8 mm_env.sv — The Verification Environment

`mm_env` is the orchestrator. Its constructor allocates the three mailboxes (`gen2drv`, `gen2scb`, `mon2scb`), creates one instance of each verification component, and passes the right handles into each constructor. The wiring done here is what gives the diagram in Section 2.1 its connectivity.

Constructor Wiring

```

gen2drv = new();    // Generator → Driver
gen2scb = new();    // Generator → Scoreboard (expected)
mon2scb = new();    // OutputMon → Scoreboard (actual)

gen     = new(gen2drv, gen2scb, num_tests);
drv     = new(vif, gen2drv);

```

```
i_mon = new(vif);
o_mon = new(vif, mon2scb);
scb   = new(gen2scb, mon2scb);
cov   = new(vif);
```

run() Task

run() forks all six component tasks in parallel using fork/join_none, then synchronizes the end-of-test by:

- waiting until both generator-side mailboxes are drained (gen2drv.num()==0 && gen2scb.num()==0);
- then waiting num_tests × (4*N + 20) clock cycles to guarantee all in-flight transactions have drained through the pipeline and been scored;
- calling scb.report() and cov.report() to print the final banners.

The choice of fork/join_none rather than fork/join is intentional: the consumer tasks (driver, monitors, scoreboard, coverage) are all forever-loops that never voluntarily return, so a join would deadlock. The environment ends them implicitly when its own run() returns and control flows back to mm_test.

3.9 mm_test.sv — The Test Class

mm_test is the highest-level user-visible test. It is intentionally small so that adding a new test (e.g., a stress test, an error-injection test, a back-to-back test) is just a matter of writing a new class with the same shape.

reset_dut()

Drives the DUT into a known state at simulation start: clears all stimulus signals, holds rst_n low for 5 cycles, releases rst_n, then waits 2 more cycles before returning.

manual_fsm_coverage()

After the env's randomized regression finishes, this task injects three deterministic sequences directly through the virtual interface (no transaction objects, no scoreboard) to close the last few code-coverage holes the constrained-random pass left behind:

- Reset asserted while the FSM is in LOAD — exercises the LOAD→IDLE async-reset transition.
- Reset asserted while the FSM is in DRAIN — exercises the DRAIN→IDLE async-reset transition.
- A clean transaction allowed to finish — exercises the natural DONE→IDLE transition.

Without these three sequences, three FSM transitions in controller.sv would remain uncovered (a controller can only be reset out of LOAD or DRAIN by the testbench actively asserting rst_n at the right moment). Doing this in a separate task keeps the random world clean and makes the intent explicit.

run()

Calls reset_dut(), constructs and runs an mm_env, then calls manual_fsm_coverage(), then prints the final "Simulation complete" banner. The order matters: the env-driven random regression must finish first so the scoreboard report and the functional coverage report reflect only legal transactions.

3.10 tb_class_mm_accelerator.sv — The Top Module

The top module is plain SystemVerilog (not a class). It does five jobs:

- Generates the 100 MHz clock (period = 10 ns).
- Instantiates the mm_if interface and connects it to the DUT (mm_accelerator_top u_dut) through dut_if.
- Hosts all four functional covergroups at module scope (because of the VCS limitation noted in Section 3.7).
- Configures the FSDB waveform dump (fsdbDumpfile / fsdbDumpvars) so make wave can replay the run in Verdi.
- In an initial block: constructs an mm_test, calls test.run(), then prints the functional coverage report and calls \$finish.

Module-level Covergroups

Covergroup	Triggered on	Bins / coverpoints
cg_a_range	@(posedge clk iff valid_in_upstream)	cp_a: zero / low / mid_low / mid_hi / high / max_val
cg_b_range	@(posedge clk iff valid_in_upstream)	cp_b: zero / low / mid_low / mid_hi / high / max_val
cg_result	@(posedge clk iff result_valid)	cp_result[15:0]: zero / sm / med / lg
cg_fsm	@(posedge clk)	cp_ready, cp_rv, cp_start + cross cp_ready×cp_rv with ignore_bins for the impossible (LOAD ∧ DONE) corner

The cross in cg_fsm uses ignore_bins to formally tell the simulator that ready=1 (LOAD) and result_valid=1 (DONE) cannot occur in the same cycle by construction — without that exclusion the cross would never reach 100% and the report would be misleading.

4. Communication and Synchronization

4.1 Mailboxes

All inter-component communication uses parameterized SystemVerilog mailboxes carrying `mm_transaction` handles. Mailboxes are unbounded (constructed with `new()` and no size argument), which means producers never block on `put()` and the only synchronization point is the consumer's `get()`, which blocks until a handle is available.

Mailbox	Producer → Consumer	Carries
gen2drv	Generator → Driver	Stimulus transaction (A, B, burst_id) — driver applies it to the DUT
gen2scb	Generator → Scoreboard	Same handle as gen2drv but used by the scoreboard for golden C[]
mon2scb	Output Monitor → Scoreboard	Captured-result transaction (got_C, burst_id)

Note that `gen2drv` and `gen2scb` both receive the same transaction handle put twice by the generator, which is safe in SystemVerilog because the mailbox stores a reference, and the transaction object is never modified after randomization.

4.2 Virtual Interface Handle (vif)

A single virtual `mm_if` handle is created in the top module (`dut_if`), passed to `mm_test`, then on to `mm_env`, and finally distributed to every component that needs to drive or sample DUT signals: driver, input monitor, output monitor, and the coverage helper. Because `vif` is a reference, all components see the same physical signals; the SystemVerilog scheduler resolves driver writes in the appropriate clocking-block region.

4.3 fork / join_none

All six component tasks (`gen.run`, `drv.run`, `i_mon.run`, `o_mon.run`, `scb.run`, `cov.run`) are launched together inside `mm_env.run()` with `fork ... join_none`. `join_none` means execution continues past the fork without waiting; the environment then explicitly waits for the empty-mailbox condition followed by a fixed pipeline-drain window before printing reports. The forked tasks are forever-loops by design — they are torn down implicitly when the static initial block in the top module returns and `$finish` is called.

5. Coverage Strategy and Closure

5.1 Functional Coverage

Four covergroups were defined to measure how well the stimulus space is exercised:

- **cg_a_range / cg_b_range** — histogram of stimulus operand values, sampled every cycle `valid_in_upstream` is high. Six bins each: zero, low [1-63], mid_low [64-127], mid_hi [128-191], high [192-254], max_val [255].
- **cg_result** — histogram of the lower 16 bits of every result element, sampled when `result_valid` pulses. Four bins: zero, sm [1-1000], med [1001-32767], lg [32768-65535].
- **cg_fsm** — FSM-state proxy histograms (ready, result_valid, start) plus a cross product that excludes the impossible ready^result_valid corner.

The deterministic seed-overrides for transactions 1–6 in the generator were specifically chosen to walk through every `cp_a` / `cp_b` / `cp_result` bin. The remaining 14 randomized transactions plus the `manual_fsm_coverage` sequence in `mm_test` then close the FSM and condition coverage.

5.2 Code Coverage

VCS is invoked with `-cm line+branch+cond+fsm+tgl`, which records all five code-coverage metrics during simulation. The `cov_dir` database is regenerated on every run; make cov merges and post-processes it with `urg` into a HTML/text report at `cov_report/`.

5.3 Final Numbers

From the captured `sim.log` of the latest regression:

Metric	Result
Total transactions run	20 randomized + 3 manual FSM sequences
Scoreboard transaction-level	20 PASS / 0 FAIL
Scoreboard element-level (4×4=16 per txn)	320 PASS / 0 FAIL
cg_a_range	100.0%
cg_b_range	100.0%
cg_result	100.0%
cg_fsm	100.0%
Overall verdict	*** ALL TESTS PASSED ***

6. Build, Run and Reproduce

All compilation and simulation flow is captured in doc/makefile. The relevant targets are:

Make Target	Purpose
make compile	Run vcs with the full file list (RTL + tb_class top), produce simv
make run	Compile then simulate with default seed (42); transcript piped to sim.log
make run SEED=<n>	Re-run with a specific +ntb_random_seed for reproducible debug
make regress N=<k>	Run k seeds back-to-back, accumulating coverage across the runs
make cov	urg the merged coverage database into cov_report/ (HTML + text)
make wave	Open the FSDB waveform in Verdi
make clean	Remove all generated artefacts

The compile is clean — no warnings or errors are reported by VCS in compile.log — which satisfies the deliverable requirement that the codes must be compile-clean.

7. Conclusion

The MS2 class-based testbench delivers a fully reusable, parameterized verification environment for the systolic-array matrix-multiply accelerator. By layering the verification logic into transaction, generator, driver, monitors, scoreboard, coverage, environment and test classes — and connecting them with mailboxes and a single virtual interface — the testbench cleanly separates stimulus from checking from coverage. A small amount of deterministic seeding in the generator and three manual reset-injection sequences in mm_test were enough to reach 100% on every functional covergroup; constrained randomization and the corner-mode distribution constraint then carry the rest of the regression. The final regression of 20 randomized transactions produced 320/320 element matches and zero scoreboard failures, demonstrating that both the design and the verification environment are functioning as intended.

The architecture is ready for two natural next steps: (1) lifting the parameters to $N = 8$ or $N = 16$ — only the localparam in the top module changes, no class needs to be touched — and (2) migrating the layered hierarchy onto UVM, where each class maps almost one-to-one onto a UVM component. Both extensions become straightforward exactly because the MS2 testbench was built in object-oriented style from the start.